The Honjo Masamune Language: A Cut-Primitive Domain-Specific Language for Cheminformatics by Individuation

Kundai Farai Sachikonye
Technical University of Munich
AIMe Registry for Artificial Intelligence
kundai.sachikonye@wzw.tum.de

June 25, 2026

Abstract

We specify *Honjo Masamune* (`honjo`; source extension `.hj`), a domain-specific language for cheminformatics in which the sole computational primitive is the *cut*: the act of individuating a part from the rest of a bounded whole at a strictly positive, irreducible cost—the *floor* $\beta > 0$. The language is built on a single semantic identity carried from its underlying theory: generating an atom, forming a chemical bond, and tracking an item through a process are not distinct operations but the same cut operation at different arities. A length-one cut individuates an atom; a cut between two items is a bond; a residue-chained sequence of cuts is a process track. honjo therefore has one verb and one composition rule, from which atom generation, compound construction, molecular geometry, and causal process tracking all follow. We give: (i) the semantic model—a finite weighted *contact graph* on which every honjo value is a cut of accountable residue, never a bare number; (ii) a complete lexical and context-free grammar in EBNF; (iii) a static *accountability* type system in which every value carries the floor at which it was cut, so that a value claiming zero residue is rejected at compile time; (iv) a small-step operational semantics in which evaluation *is* measurement (executing a cut performs it; it does not simulate it), and in which the committed-cut count is monotone, giving every program an intrinsic, non-decreasing clock; (v) a standard library of verbs, each bound by name to a validated operation of the underlying framework (`cut` $\rightarrow$ spectroscopic atom individuation; $\sim$ and `close` $\rightarrow$ boundary-thickness bonding and shell closure; `track...until...yield` $\rightarrow$ accountable propagation with S-entropy convergence admissibility); and (vi) a two-target compilation model. honjo compiles, from one front end and one typed core intermediate representation, to two back ends: a TypeScript/WebGL 2 runtime for the interactive online tool, in which a program's cuts are realised as GPU render passes (rendering is measurement), and a Rust runtime for the reference implementation, in which cuts are realised as exact maximum-flow minimum-cut computations on the contact graph. We prove that the two back ends are observationally equivalent up to the floor: any honjo program yields the same accountable result on both, differing only below the resolution the floor already forbids. The specification is implementable as written; a companion reference interpreter realises the operational semantics directly.

Keywords: domain-specific language; cheminformatics; individuation; cut primitive; contact graph; accountability type system; operational semantics; minimum cut; rendering-as-measurement; two-target compilation; TypeScript; Rust.

Contents

1 Introduction

1.1 Papers as programs

The framework underlying this language consists of a sequence of results that are, in form, *operations*: a procedure that individuates an atom from spectroscopic structure; a procedure that decides whether two atoms bond and in what geometry; a procedure that tracks an item's uncertainty through a process and returns an accountable account of what happened to it. Each such result is already, in effect, a program with a proof of correctness attached. This specification makes that literal: it defines a language whose statements *are* those operations, so that a paper becomes a script and a derivation becomes a run.

The design discipline throughout is that we implement only what is already established. Every verb of the language binds to an operation of the framework that has an explicit construction and has been validated numerically; the language adds composition, typing, and execution, not new physics or chemistry.

1.2 One primitive: the cut

The language has a single computational primitive, the *cut*, and a single reason for it. To individuate any part of a bounded whole—to make it a distinct thing—costs a strictly positive, irreducible minimum, the floor $\beta > 0$. This one fact is the engine of the underlying theory, and it is the engine of the language. Every honjo operation is a cut:

- A *generation* is a cut of arity one: individuating an atom of atomic number Z from the medium (Section 7.1).
- A *bond* is a cut of arity two: the single shared boundary between two items, which exists iff joining lowers total boundary thickness (Section 7.2).
- A *compound* is a closure of cuts: cuts continued until every constituent's vacancy is driven to zero (Section 7.3).
- A *track* is a residue-chained sequence of cuts: the residue of each cut is the cause of the next, and the chain is the propagation of an item's uncertainty through a process (Section 7.4).

Generation is not a different verb from tracking; it is a track of length one. This is the central design decision and it is forced by the theory, not chosen for convenience: in the underlying framework a reaction step and a measurement step are the same cut, and generation, bonding, and tracking are that cut at arities one, two, and many. honjo therefore has one verb (`cut`) and one composition rule (residue chaining); all other surface forms are sugar over these.

1.3 Two targets, one language

honjo exists in two parallel implementations from a single specification:

1. A **TypeScript** compiler and WebGL 2 runtime for the interactive online tool. Here a cut is realised as a GPU render pass: the framebuffer that results from rendering *is* the measurement, in keeping with the rendering–measurement identity of the underlying theory. This is the fast, exploratory, in-browser version.
2. A **Rust** compiler and native runtime for the reference and production version. Here a cut is realised as an exact maximum-flow minimum-cut computation on the contact graph. This is the authoritative version against which the online tool is checked.

Both share one front end (lexer, parser, type checker) and one typed core intermediate representation (Section 8); they differ only in the back end that realises a cut. We prove (Theorem 8.4) that the two back ends agree on every program up to the floor—the one resolution at which the theory already forbids finer distinction—so the online tool and the reference implementation never disagree about anything the language can express.

1.4 What this document specifies

Section 2 fixes the semantic model: the contact graph, the cut, the floor, the residue, and accountability. Section 3 gives the lexical grammar; Section 4 the context-free grammar in EBNF. Section 5 defines the accountability type system and proves its soundness. Section 6 gives the small-step operational semantics and proves cut monotonicity and measurement-not-simulation. Section 7 specifies the standard library, binding each verb to its framework operation.

Section 8 defines the core IR and the two-target compilation model and proves target equivalence. Section 9 gives worked programs. Section 10 states implementation requirements for both targets. Throughout, framework operations are named and their input/output contracts stated; their derivations are external to this document.

2 The semantic model

honjo is defined over a single mathematical object, the contact graph, and a single operation on it, the cut. We fix these before any syntax.

2.1 The contact graph

Definition 2.1 (Contact graph)**.** A *contact graph* is a finite weighted graph $\Gamma = (V, E, w)$ with a distinguished *medium* vertex $\mathsf{m} \in V$ adjacent to every other vertex, and $w\colon E \to \mathbb{R}_{>0}$ a strictly positive weight. Vertices $v \neq \mathsf{m}$ are *items*; an edge between two items is a *contact*; $w(e)$ is the cost of the separation it maintains. The graph is the runtime state of a honjo program: every value the program holds is a structure on Γ.

Definition 2.2 (Floor)**.** The *floor* of a contact graph is a constant $\beta > 0$ with $w(e) \geq \beta$ for every edge. No separation is free. The floor is a program-level parameter (set by `floor`, Section 4); its positivity is invariant and enforced by the type system (Section 5).

Definition 2.3 (Cut)**.** A *cut* of an item set S ($\varnothing \neq S \subsetneq V$, $\mathsf{m} \notin S$) is the edge boundary $\partial(S) = \{e \in E : |e \cap S| = 1\}$, of weight $w(\partial(S)) = \sum_{e \in \partial(S)} w(e) \geq \beta$. The cut individuates S from the rest. A cut *event* commits one such boundary: it fixes the edges of $\partial(S)$ as realised separations and advances the program's cut count by one.

Definition 2.4 (Residue)**.** The *residue* of a cut event is the weight $\rho = w(\partial(S)) \geq \beta$ it deposits as committed boundary. The residue is the value the cut produces and the cause of any subsequent cut: a chain of cuts is one in which each residue is consumed as the boundary condition of the next (Section 7.4).

Principle 2.5 (Accountability)**.** Every honjo value is a cut, and every cut carries its residue. There are no bare numbers in honjo: a quantity is always paired with the floor at which it was individuated. A value claiming residue below the floor, or zero residue, is not a value—it is the forbidden sharp cut, and the type system rejects it (Theorem 5.4).

Remark 2.6 (Why a graph and not a number line). honjo computes on a contact graph rather than on real numbers because the quantities it manipulates—atomic identity, bond existence, process accountability—are minimum cuts, not scalars. A bond's strength is the weight of a shared boundary; an atom's identity is the invariant minimum cut against the medium; a track's result is an accountable chain of cuts. Representing these as graph cuts (rather than as fitted scalars) is what lets the same primitive serve generation, bonding, and tracking, and is what the two back ends each realise: exactly (Rust, max-flow) or by rendering (TypeScript, GPU).

2.2 Measurement, not simulation

Principle 2.7 (Evaluation is measurement)**.** Executing a honjo cut performs the individuation; it does not simulate a description of it. The result of `cut 8` is the measured partition structure of oxygen, carried as an accountable value, not a numerical model of oxygen. Operationally (Section 6) this means a cut evaluation mutates the contact graph—it commits boundary and advances the count—rather than returning a pure function of inputs. The distinction is observable: re-evaluating a cut is a new cut at a higher count, never a cached recomputation (Theorem 6.2).

3 Lexical structure

A honjo source file is UTF-8 text with extension `.hj`. Lexing produces a token stream from the following classes.

Definition 3.1 (Token classes)**.**

- **Comments.** `-` to end of line; discarded.
- **Keywords.** `floor`, `cut`, `close`, `track`, `until`, `yield`, `when`, `do`, `emit`, `observe`, `in`, `as`, `let`, `medium`, `converge`, `diverge`, `with`, `by`, `import`, `module`, `export`, `assert`.
- **Identifiers.** `[A-Za-z_][A-Za-z0-9_]*`, not a keyword.
- **Numbers.** decimal integers and floats with optional exponent: `[0-9]+(.[0-9]+)?([eE][+-]?[0-9]+)?`.
- **Strings.** double-quoted, with the usual escapes.
- **Operators and punctuation.** `:=` (bind), `~` (cut between / bond), `( ) [ ] { } , : . > < >= <= == ->` and the range/bound forms used by `cell`/`bounds`.

Whitespace is insignificant except as a token separator; honjo is not layout-sensitive.

Notation 3.2 (Floors and units in literals). A numeric literal may carry an explicit floor annotation with the suffix form `value#floor` (read "value at floor"); e.g. `1.0e-7#1e-9`. A literal without an explicit floor inherits the ambient program floor set by the most recent `floor` declaration in scope. There is no way to write a literal of zero floor (Theorem 5.4).

4 Grammar

We give the context-free grammar in EBNF (International Organization for Standardization, 1996). Nonterminals are *italicised*; terminals are `typewriter`; `{x}` is zero or more, `[x]` optional, `|` alternation.

Definition 4.1 (honjo grammar)**.**

program ::= {*decl*}
decl ::= *floorDecl* | *import* | *moduleDecl* | *stmt*
floorDecl ::= `floor` *number*
import ::= `import` *qname*
moduleDecl ::= `module` *ident* `{` {*decl*} `}`
stmt ::= *bind* | *track* | *observe* | *assert* | *expr*
bind ::= [`let`] *ident* `:=` *expr*
expr ::= *cut* | *bondExpr* | *closeExpr* | *call* | *ident* | *number* | *string* | `(` *expr* `)`
cut ::= `cut` *cutArg*
cutArg ::= *number* | *ident* | *ident* `~` *ident*
bondExpr ::= *expr* `~` *expr* [`when` *cond*]
closeExpr ::= `close` *ident* `(` *argList* `)` [`by` *ident*]
track ::= `track` *ident* `in` *expr*
[`with reps` *repList*]
`until` *admit* `yield` *ident*
admit ::= `converge` | `diverge` | *cond*
observe ::= `observe` *expr* [`as` *ident*]
assert ::= `assert` *cond* [`emit` *string*]
call ::= *qname* `(` [*argList*] `)`
argList ::= *arg* {`,` *arg*}
arg ::= [*ident* `:`] *expr*
cond ::= *expr relop expr*
relop ::= `>` | `<` | `>=` | `<=` | `==`
repList ::= *ident* {`,` *ident*}
qname ::= *ident* {`.` *ident*}
number ::= *numlit* [`#` *numlit*]

Remark 4.2 (Reading the core forms). The whole language is small. `cut N` generates (arity-one cut); `a ~ b when c` bonds (arity-two cut, guarded); `close X(...)` drives an item to shell closure (cut-to-closure); `track x in P until converge yield r` propagates and binds the accountable result. `observe` forces measurement of a value (commits its cut now); `assert` is a pre-flight check that may halt with a message. Everything else—`let`, `import`, `module`, named arguments—is conventional.

5 The accountability type system

honjo's types record *what kind of cut* a value is and *at what floor*. The system's purpose is to make Principle 2.5 statically enforceable: a value of zero residue cannot be typed.

5.1 Types

Definition 5.1 (Types)**.** The honjo types are

$$\tau ::= \mathsf{Atom} \mid \mathsf{Bond} \mid \mathsf{Compound} \mid \mathsf{Path} \mid \mathsf{Scalar} \mid \mathsf{Cut}.$$

Cut is the supertype of all individuated values; Atom, Bond, Compound, Path are its arity- and role-refinements (Atom = arity-one cut, Bond = arity-two cut, Compound = closure of cuts, Path = residue chain). Scalar is a measured number; it is never bare. Every typed value v additionally carries a *floor annotation* $\beta(v) > 0$ and a *residue* $\rho(v) \geq \beta(v)$.

Definition 5.2 (Typing judgement)**.** A judgement $\Gamma \vdash e : \tau \mathbin{@} \beta$ reads "in floor context Γ (which fixes the ambient floor), expression e has type τ and is individuated at floor $\beta > 0$." The context carries the ambient floor set by the most recent `floor` declaration and the types of bound identifiers.

5.2 The positivity rule

Rule 5.3 (Floor positivity)**.** Every value-introducing rule has the side condition $\beta > 0$ and $\rho \geq \beta$. A literal or expression whose annotation forces $\beta \leq 0$ or $\rho < \beta$ is ill-typed.

$$\frac{\Gamma \vdash \mathit{numlit}\, n \quad \beta > 0}{\Gamma \vdash n\#\beta : \mathsf{Scalar} \mathbin{@} \beta} \qquad \frac{}{\Gamma \vdash \texttt{cut}\ Z : \mathsf{Atom} \mathbin{@} \beta_\Gamma}$$

where β_Γ is the ambient floor.

Theorem 5.4 (No zero-residue value)**.** *There is no well-typed honjo expression of residue* 0*. Equivalently, the sharp cut is not expressible: every typeable value has* $\rho \geq \beta > 0$*.*

Proof. By induction on typing derivations. The only value introductions are literals (Theorem 5.3, side condition $\beta > 0$, and a literal's residue is its annotated floor), cuts (residue $= w(\partial(S)) \geq \beta > 0$ by Theorem 2.3), and the derived forms (bond, close, track) whose residues are sums or chains of cut residues, hence $\geq \beta$. No rule introduces a value of residue $< \beta$, and $\beta > 0$ is a side condition on every floor-introduction. Hence no derivation concludes $\rho = 0$. □

5.3 Cut typing

Rule 5.5 (Generation; bond; close)**.**

$$\frac{\Gamma \vdash Z : \mathsf{Scalar} \quad Z \in \mathbb{N}^+}{\Gamma \vdash \texttt{cut}\ Z : \mathsf{Atom} \mathbin{@} \beta_\Gamma} \qquad \frac{\Gamma \vdash a : \mathsf{Atom} \quad \Gamma \vdash b : \mathsf{Atom}}{\Gamma \vdash a \sim b : \mathsf{Bond} \mathbin{@} \beta_\Gamma}$$

$$\frac{\Gamma \vdash X : \mathsf{Atom} \quad \Gamma \vdash a_i : \mathsf{Atom}}{\Gamma \vdash \texttt{close}\ X(a_1, \ldots, a_k) : \mathsf{Compound} \mathbin{@} \beta_\Gamma}$$

The bond rule additionally requires its guard, when present, to be a Bool-valued *cond*; an unguarded bond is admitted only if static analysis cannot exclude $\Delta\text{thick} > 0$ (Section 7.2).

Rule 5.6 (Track)**.**

$$\frac{\Gamma \vdash x : \mathsf{Atom} \text{ or } \mathsf{Compound} \quad \Gamma \vdash P : \mathsf{Compound} \text{ or } \mathsf{Path}}{\Gamma \vdash \texttt{track}\ x\ \texttt{in}\ P\ \texttt{until}\ \mathit{admit}\ \texttt{yield}\ r : \mathsf{Path} \mathbin{@} \beta_\Gamma}$$

The bound result r has type Path; its residue is the total committed boundary of the propagation (Section 7.4).

Theorem 5.7 (Type soundness)**.** *honjo typing is sound for the operational semantics of Section 6: if* $\Gamma \vdash e : \tau \mathbin{@} \beta$ *and* e *evaluates to a value* v*, then* v *has type* τ*, floor* $\beta > 0$*, and residue* $\rho(v) \geq \beta$ *(progress and preservation).*

Proof sketch. Standard syntactic soundness (Pierce, 2002). *Progress*: a well-typed closed expression is a value or steps (every form has a reduction rule in Section 6). *Preservation*: each reduction rule preserves the type and the floor annotation, and—by Theorem 5.4 applied at each step—preserves $\rho \geq \beta > 0$. The only non-standard clause is that reductions are state-mutating (measurement, Theorem 2.7); preservation holds because the cut count and committed boundary are monotone (Theorem 6.2), so no step lowers a residue below the floor. □

6 Operational semantics

We give a small-step semantics over configurations $\langle \Gamma_G, M, e \rangle$, where Γ_G is the current contact graph (Theorem 2.1), $M \in \mathbb{N}$ is the committed-cut count (the program clock), and e is the expression under evaluation. We write $\langle \Gamma_G, M, e \rangle \to \langle \Gamma'_G, M', v \rangle$ for one step. Evaluation is measurement: steps mutate Γ_G and increment M.

6.1 Cut reduction

Rule 6.1 (Cut commits and counts)**.**

$$\frac{S = \textsc{Individuate}(\Gamma_G, Z) \qquad \rho = w(\partial(S)) \geq \beta}{\langle \Gamma_G, M, \texttt{cut } Z \rangle \to \langle \Gamma_G \oplus \partial(S),\ M+1,\ v_{\mathsf{Atom}}(S, \rho) \rangle}$$

where Individuate is the framework's atom-individuation operation (Section 7.1), $\Gamma_G \oplus \partial(S)$ is Γ_G with the cut's boundary committed, and $v_{\mathsf{Atom}}(S, \rho)$ is the resulting accountable atom value.

The bond, close, and track rules are analogous: each commits boundary and increments M by the number of cut events it performs (one for a bond, the closure count for `close`, the chain length for `track`).

Theorem 6.2 (Cut monotonicity / intrinsic clock)**.** *For any program, the committed-cut count M is non-decreasing along evaluation and strictly increases at every cut event. No reduction decreases M. Consequently re-evaluating the same surface expression is a new cut at a strictly higher count, never a cached recomputation, and two configurations with identical graphs but different M are distinct states.*

Proof. Every value-producing rule (Theorem 6.1 and its analogues) increments M; no rule decrements it (there is no reduction that un-commits boundary—"undo" is itself a further cut). Hence M is monotone and strictly increases per cut event. Distinctness of equal-graph states at different M is immediate since M is part of the configuration. □

Corollary 6.3 (Measurement, not simulation)**.** *Because evaluation mutates Γ_G and advances M, a honjo cut cannot be a pure function memoised across calls: each occurrence performs the individuation afresh and is recorded in the clock. This realises Principle 2.7: running the program* is *performing the measurements it names.*

6.2 Track reduction and convergence

Rule 6.4 (Track is a residue chain)**.**

$$\frac{\begin{array}{c}(S_1, \ldots, S_k) = \textsc{Propagate}(\Gamma_G, x, P) \quad \rho_i = w(\partial(S_i)) \\ \textsc{Admit}(\rho_1, \ldots, \rho_k) = \mathit{admit}\end{array}}{\langle \Gamma_G, M, \texttt{track } x \texttt{ in } P \texttt{ until } \mathit{admit} \texttt{ yield } r \rangle \to \langle \Gamma'_G,\ M+k,\ v_{\mathsf{Path}}(S_{1:k}, \textstyle\sum_i \rho_i) \rangle}$$

Propagate is the framework's accountable-propagation operation; each ρ_i is the residue of cut i and the boundary condition of cut $i{+}1$ (residue chaining). Admit tests the convergence criterion (S-entropy convergence for `converge`); a non-admissible chain reduces to the empty path (no yield).

Remark 6.5 (Local steps are free; only convergence is judged)*.* In keeping with the underlying propagation theory, the intermediate cuts of a track are unconstrained: a step may be locally extreme (even "impossible") provided the chain composes to a value that meets the admissibility criterion. Admit therefore inspects only the terminal convergence of the chain, not the plausibility of intermediate ρ_i. This is what makes `track...until converge` a search over admissible propagations rather than a single forced path.

7 The standard library

Each honjo verb binds to a named operation of the underlying framework. We state the contract (inputs, output, the operation it invokes); the operation's construction and validation are external to this document.

7.1 `cut` — atom individuation

Definition 7.1 (Individuate). `cut Z` invokes Individuate(Z): from atomic number $Z \in \mathbb{N}^+$, fill the partition coordinates (n, ℓ, m, s) under the shell capacity $C(n) = 2n^2$, exclusion, and energy ordering, returning an Atom value carrying the configuration, the valence-shell occupancy q_v and capacity C_v, the vacancy $\nu = C_v - q_v$, the ground term symbol, and the residue $\rho = w(\partial(\{Z\})) \geq \beta$. This is the spectroscopic atom-derivation operation; e.g. `cut 6` yields carbon [He]$2s^2 2p^2$, ${}^3P_0$, $\nu = 4$.

7.2 $\sim$ — bonding

Definition 7.2 (Bond). `a ~ b` invokes Bond(a, b): form the shared-boundary cut between atoms a, b and return a Bond value iff the shared content $\Delta\text{thick}(a, b) > 0$ (joint individuation is cheaper than separate), bounded by $\min(\nu_a, \nu_b)$; otherwise no bond. The optional guard `when` c requires c to hold (e.g. `when delta > 0`). This is the boundary-thickness bonding criterion; a closed-shell atom ($\nu = 0$) bonds with nothing.

7.3 `close` — compound by closure

Definition 7.3 (Close). `close X(`$\mathtt{a}_1$`,...,`$\mathtt{a}_k$`)` invokes Close: form the set of bonds that drives every constituent's vacancy to zero, returning a Compound value carrying the stoichiometry (fixed by vacancy matching), the per-centre geometry (the maximal angular separation of the committed interfaces, e.g. 109.47° tetrahedral, compressed by unshared regions), and total residue. E.g. `close O(H,H)` yields a bent 2:1 water at $\approx 104.5°$.

7.4 `track` — accountable propagation

Definition 7.4 (Propagate, Admit). `track x in P until converge yield r` invokes Propagate(x, P): track item x's individuation through process P as a residue-chained sequence of cuts, and Admit the chain iff it S-entropy-converges to the output (the only admissibility test; local steps free, Theorem 6.5). The yielded Path is the *amalgamation*: the accountable set of cuts—reaction steps and measurement steps indifferently—by which x reached the output. Optional `with reps` R permits representation switching between steps (mean-recovery-equivalent decompositions) at no additional cut cost.

Remark 7.5 (The verbs are one verb). Individuate, Bond, Close, and Propagate are the cut at arities one, two, closure, and chain. The standard library is therefore not four independent operations but one operation (the cut) exposed at four arities, exactly as Section 1 claims and as the underlying theory requires.

8 Core IR and the two-target model

honjo compiles, from one front end, to a typed core intermediate representation, and thence to two back ends. This section fixes the IR and proves the two back ends agree.

8.1 The core IR

Definition 8.1 (Cut-IR)**.** *Cut-IR* is a typed sequence of cut instructions over a contact-graph state. Each instruction is one of: $\textsc{Ind}(Z)$ (individuate), $\textsc{Bnd}(a, b, g)$ (guarded bond), $\textsc{Cls}(X, \bar{a})$ (close), $\textsc{Prp}(x, P, adm)$ (propagate), $\textsc{Obs}(v)$ (force measurement). Each instruction carries its result type and floor (Section 5) and, when executed, returns an accountable value and increments the cut count. Cut-IR is back-end-agnostic: it names cuts, not how they are realised.

The front end (lexer, parser, type checker of Section 3–5) is shared by both targets and lowers a program to Cut-IR. Only the realisation of a cut instruction differs between targets.

8.2 Target R (Rust): cuts as minimum cuts

Definition 8.2 (Rust realisation)**.** In the Rust back end, each Cut-IR instruction is realised by an exact computation on the contact graph: Ind, Bnd, Cls compute the relevant minimum cut by maximum flow (Cormen et al., 2009; Ford and Fulkerson, 1956); Prp enumerates and evaluates admissible propagations exactly; residues are exact edge-weight sums. This is the reference semantics: it computes $w(\partial(\cdot))$ to the floating-point precision of the host.

8.3 Target T (TypeScript/WebGL): cuts as render passes

Definition 8.3 (TypeScript realisation)**.** In the TypeScript/WebGL 2 back end, each Cut-IR instruction is realised as a GPU render pass over the contact-graph state encoded in textures: a fragment shader evaluates the cut at each texel and writes the framebuffer, whose contents *are* the accountable result (rendering–measurement identity). Minimum cuts are computed by the GPU-resident flow/observation kernels; residues are read back from the framebuffer. This is the interactive online realisation.

8.4 Target equivalence

Theorem 8.4 (Two-target equivalence up to the floor)**.** *Let p be a well-typed honjo program with ambient floor β. Let v_R and v_T be the values it yields under the Rust and TypeScript back ends respectively. Then v_R and v_T have the same type and the same cut structure, and their residues agree up to the floor:*

$$|\rho(v_R) - \rho(v_T)| \;\leq\; \beta.$$

The two back ends never disagree about any distinction honjo can express, because honjo cannot express a distinction finer than β (Theorem 5.4).

Proof. Both back ends realise the same Cut-IR (Theorem 8.1), so they perform the same sequence of cut instructions with the same types and the same cut count (the front end is shared). They differ only in how each cut's residue is computed: Rust by exact max-flow, TypeScript by GPU render-readback. The GPU realisation computes the same minimum cut to its texel/precision resolution; its readback error is bounded by the coarsest resolvable boundary, which is the floor (no honjo value is individuated below β, Theorem 5.4). Hence the residues agree to within β, and since honjo admits no sub-floor distinction, the two results are observationally identical at the language level. □

Remark 8.5 (Why two targets, not one). The split is deliberate and the equivalence theorem is what makes it safe. The TypeScript/WebGL target is fast, in-browser, and renders cuts as measurements directly—ideal for the interactive sandbox. The Rust target is exact and authoritative—ideal as the reference against which the online tool is validated. Theorem 8.4 guarantees the online tool is never *wrong* relative to the reference; it is at most less precise, and only below the floor, where the theory forbids meaning anyway.

9 Worked programs

9.1 Generate an atom

Listing 1: Individuating carbon (a length-one cut).

```
-- carbon.hj
floor 1.0 -- set the ambient floor; nothing is free

C := cut 6 -- individuate Z=6 against the medium
observe C -- force the measurement now
-- C : Atom @ 1.0
-- config [He] 2s2 2p2
-- term 3P_0
-- vacancy 4
```

9.2 Build a compound

Listing 2: Water: cuts to closure, geometry by separation.

```
-- water.hj
floor 1.0

O := cut 8
H := cut 1

-- a bond is a cut between two items, admitted only if it lowers thickness
OH := O ~ H when delta > 0

-- close drives every vacancy to zero: stoichiometry + geometry follow
W := close O(H, H) -- 2:1, bent, ~104.5 deg
observe W
assert W.valence == closed emit "water did not close"
```

9.3 Run the causal table

Listing 3: Tracking an item through a process.

```
-- track.hj
floor 1.0
import honjo.causal

O := cut 8
W := close O(H, H)

-- propagate O's uncertainty through the process; admissible iff it
-- S-entropy-converges to the observed output
path := track O in W
        with reps mass, charge, time -- representation switching allowed
        until converge
        yield amalgamation

observe path -- the amalgamation IS the result
```

Remark 9.1 (The same keyword, three jobs). Across the three programs, `cut` generates, `~` (a cut between) bonds, `close` (cuts to closure) builds, and `track` (a residue chain of cuts) propagates—all the same primitive at different arities (Theorem 7.5). A reader who understands the cut understands the whole language.

10 Implementation requirements

Both targets must implement, from this specification:

1. The shared front end: lexer (Section 3), parser to the grammar (Theorem 4.1), and accountability type checker (Section 5) enforcing Theorem 5.4.
2. Lowering to Cut-IR (Theorem 8.1).
3. The operational semantics of Section 6: a contact-graph state, a monotone cut count, and the cut/bond/close/track reductions, with evaluation mutating state (measurement, Theorem 6.3).
4. The standard-library bindings of Section 7 to the framework operations INDIVIDUATE, BOND, CLOSE, PROPAGATE.
5. A back end: exact max-flow minimum cut (Rust, Theorem 8.2) or GPU render-pass realisation (TypeScript/WebGL, Theorem 8.3).

A conformance suite must check Theorem 8.4: every example program yields type- and structure-identical results on both targets with residues agreeing to within the floor.

Remark 10.1 (Reference interpreter). A direct interpreter of Section 6 over the exact (max-flow) back end constitutes the simplest conforming implementation and serves as the oracle for both production targets. It can reuse the already-validated framework operations directly as the standard-library bindings.

11 Conclusion

We have specified Honjo Masamune as a language whose single primitive is the cut—the act of individuating a part of a bounded whole at the irreducible cost $\beta > 0$—and whose every operation, from generating an atom to tracking an item through a process, is that primitive at a different arity. The accountability type system makes the floor a static invariant: no honjo value can claim the forbidden sharp cut. The operational semantics makes evaluation measurement rather than simulation, with a monotone cut count as the program's intrinsic clock. The standard library binds each verb to a validated operation of the underlying framework, so a honjo program is an executable paper. And the two-target model—one shared, typed front end lowering to a back-end-agnostic Cut-IR, realised exactly in Rust and by GPU rendering in TypeScript—gives both an authoritative reference and an interactive online tool that, by Theorem 8.4, never disagree about anything the language can express. The specification is complete and implementable as written; the reference interpreter of Theorem 10.1 realises it directly from the validated framework operations.

References

T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.

L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399–404, 1956. doi: 10.4153/CJM-1956-045-5.

International Organization for Standardization. *ISO/IEC 14977: Information technology — Syntactic metalanguage — Extended BNF*. ISO, 1996.

B. C. Pierce. *Types and Programming Languages*. MIT Press, Cambridge, MA, 2002.